# DP Computer Science: B 3.2.5 Design Patterns in OOP (HL Only)

## 1. Core Concepts & Learning Objectives

### Learning Intentions

By the end of this lesson, students will be able to:

- Explain the **purpose and benefits** of using design patterns in object-oriented programming
- Describe the key characteristics and typical use cases for the **Singleton, Factory, and Observer** design patterns
- Identify scenarios where the **Singleton, Factory, or Observer** design patterns could be applied to solve recurring programming challenges

### Key Vocabulary

| Term | Definition |
|---|---|
| **Design pattern** | A reusable solution to a commonly occurring problem in software design |
| **Singleton pattern** | Ensures a class has only one instance and provides a global access point to it |
| **Factory pattern** | Provides an interface for creating objects in a superclass, allowing subclasses to alter the type of objects created |
| **Observer pattern** | Defines a one-to-many dependency between objects so that when one object changes state, all dependents are notified |
| **Creational pattern** | A design pattern that deals with object creation mechanisms (e.g., Factory, Singleton) |
| **Structural pattern** | A design pattern that deals with how classes and objects are composed |
| **Behavioural pattern** | A design pattern that deals with communication between objects (e.g., Observer) |
| **Loose coupling** | A design principle where objects have minimal dependencies on each other |

### Relevant ATL Skills

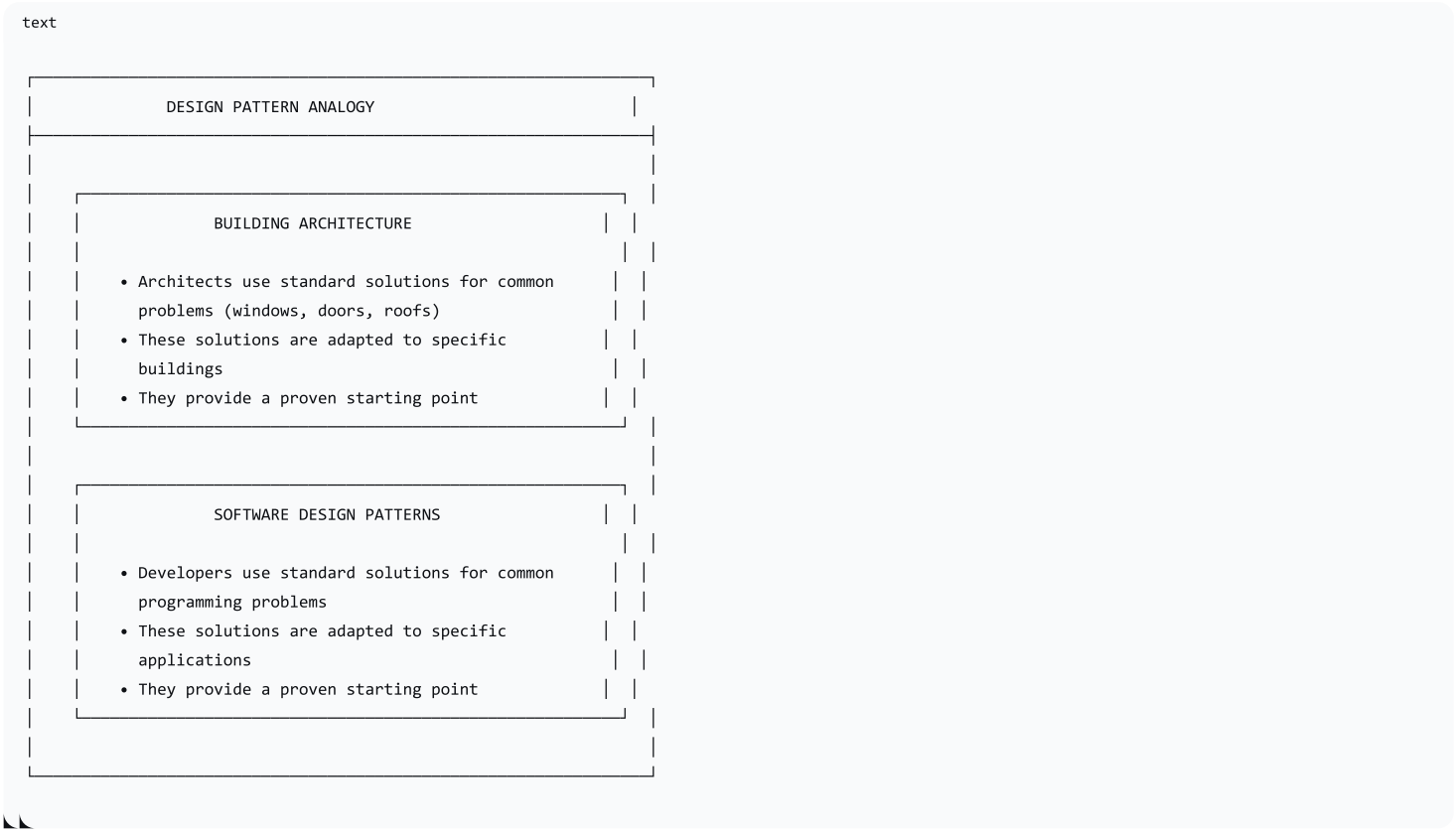
| Skill | Description |
|---|---|
| **Thinking** | Critical thinking (analysing and evaluating); transfer (applying to new scenarios); abstraction (understanding high-level solutions) |
| **Communication** | Explaining ideas clearly; using appropriate terminology |
| **Self-Management** | Organisation (structuring understanding of design patterns) |

## 2. Introduction to Design Patterns

## What are Design Patterns?

> Design patterns are reusable solutions to commonly occurring problems in software design. They are like tried-and-tested blueprints that you can adapt to solve specific problems.

## The Architectural Analogy




## Why Use Design Patterns?

```
│    │    Proven solutions to common problems                │  │
│    └──────────────────────────────────────────────────────┘  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

---

## 3. Singleton Pattern

### What is the Singleton Pattern?

> **The Singleton pattern ensures that a class has only one instance and provides a global access point to that instance.**

### Key Characteristics

| Characteristic | Description |
|---|---|
| **Single Instance** | Only one instance of the class can exist |
| **Global Access** | The instance is accessible from anywhere |
| **Private Constructor** | Prevents direct instantiation |
| **Lazy Initialisation** | Instance is created only when needed |

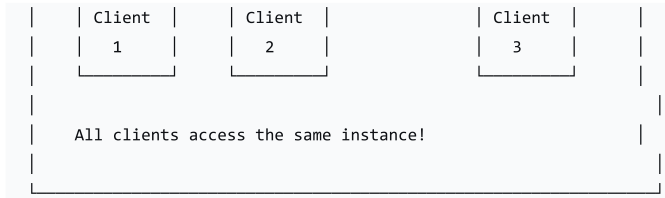
### When to Use Singleton

| Scenario | Why Singleton? |
|---|---|
| **Logging System** | Need a single logging instance throughout the application |
| **Configuration Manager** | Need a single source of configuration data |
| **Database Connection Pool** | Need a single connection pool for efficiency |
| **Printer Spooler** | Need a single point of control for printing |

### Singleton Pattern Diagram

text

```
┌──────────────────────────────────────────────────────────────┐
│             SINGLETON PATTERN DIAGRAM                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ┌──────────────────────────────────────────────────┐      │
│    │              SingletonClass                      │  │   │
│    │    ──────────────────────────────────────────    │  │   │
│    │    - instance: SingletonClass                    │  │   │
│    │    - SingletonClass()                            │  │   │
│    │    ──────────────────────────────────────────    │  │   │
│    │    + getInstance(): SingletonClass               │  │   │
│    │    + doSomething()                               │  │   │
│    └──────────────────────────────────────────────────┘      │
│                            │                                 │
│                            │                                 │
│    ┌───────────────────────┼──────────────────────┐          │
│    │                       │                      │          │
│    ▼                       ▼                      ▼          │
│    ┌─────────┐     ┌─────────┐              ┌─────────┐      │
```




## Conceptual Implementation

python

```python
# Python
class Logger:
    _instance = None  # Private class variable

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            print("Creating Logger instance")
        return cls._instance

    def log(self, message):
        print(f"Log: {message}")

# Usage
logger1 = Logger()
logger2 = Logger()
print(logger1 is logger2)  # True (same instance)
logger1.log("Hello")
logger2.log("World")
```

java

```java
// Java
public class Logger {
    private static Logger instance = null;

    private Logger() {
        // Private constructor
        System.out.println("Creating Logger instance");
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("Log: " + message);
    }
}

// Usage
Logger logger1 = Logger.getInstance();
Logger logger2 = Logger.getInstance();
System.out.println(logger1 == logger2);  // true
logger1.log("Hello");
logger2.log("World");
```

### Singleton in Action

```python
# Python: Configuration Manager
class ConfigManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.settings = {}
        return cls._instance

    def set(self, key, value):
        self.settings[key] = value

    def get(self, key):
        return self.settings.get(key)

# Usage
config1 = ConfigManager()
config2 = ConfigManager()
config1.set("db_host", "localhost")
print(config2.get("db_host"))  # "localhost"
```

---

## 4. Factory Pattern

### What is the Factory Pattern?

> The Factory pattern provides an interface for creating objects in a superclass, allowing subclasses to alter the type of objects created. It decouples the client code from the concrete classes.

### Key Characteristics

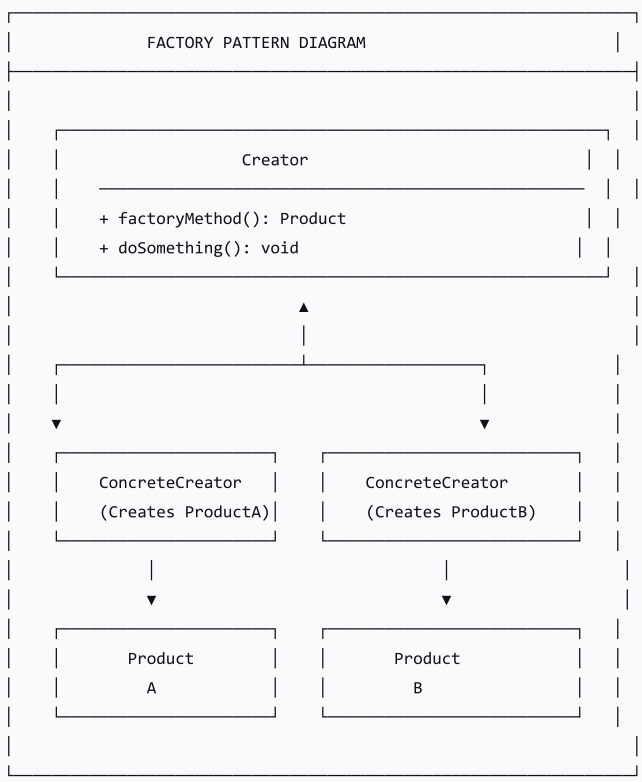
| Characteristic | Description |
|---|---|
| **Encapsulation** | Object creation logic is encapsulated |
| **Decoupling** | Client code is decoupled from concrete classes |
| **Flexibility** | Easy to add new object types |
| **Single Responsibility** | Creation logic is separated from business logic |

### When to Use Factory

| Scenario | Why Factory? |
|---|---|
| **Different Payment Types** | Creating payment processors (credit card, PayPal, crypto) |
| **UI Components** | Creating different types of buttons, windows |
| **Document Formats** | Creating different document types (PDF, Word, HTML) |
| **Database Connections** | Creating connections to different database systems |

## Factory Pattern Diagram

text




## Conceptual Implementation

python

```python
# Python
from abc import ABC, abstractmethod

# Product interface
class Payment(ABC):
    @abstractmethod
    def process(self, amount):
        pass

# Concrete Products
class CreditCardPayment(Payment):
    def process(self, amount):
        print(f"Processing ${amount} via Credit Card")

class PayPalPayment(Payment):
    def process(self, amount):
        print(f"Processing ${amount} via PayPal")

class CryptoPayment(Payment):
    def process(self, amount):
        print(f"Processing ${amount} via Cryptocurrency")

# Factory
class PaymentFactory:
    @staticmethod
    def create_payment(payment_type):
        if payment_type == "credit_card":
            return CreditCardPayment()
```

```
        elif payment_type == "paypal":
            return PayPalPayment()
        elif payment_type == "crypto":
            return CryptoPayment()
        else:
            raise ValueError("Invalid payment type")

# Usage
factory = PaymentFactory()
payment1 = factory.create_payment("credit_card")
payment1.process(100)  # Processing $100 via Credit Card
```

java

```
// Java
interface Payment {
    void process(double amount);
}

class CreditCardPayment implements Payment {
    public void process(double amount) {
        System.out.println("Processing $" + amount + " via Credit Card");
    }
}

class PayPalPayment implements Payment {
    public void process(double amount) {
        System.out.println("Processing $" + amount + " via PayPal");
    }
}

class CryptoPayment implements Payment {
    public void process(double amount) {
        System.out.println("Processing $" + amount + " via Cryptocurrency");
    }
}

class PaymentFactory {
    public static Payment createPayment(String type) {
        switch (type) {
            case "credit_card": return new CreditCardPayment();
            case "paypal": return new PayPalPayment();
            case "crypto": return new CryptoPayment();
            default: throw new IllegalArgumentException("Invalid type");
        }
    }
}

// Usage
Payment payment = PaymentFactory.createPayment("paypal");
payment.process(50.0);
```

---

## 5. Observer Pattern

### What is the Observer Pattern?

> The Observer pattern defines a one-to-many dependency between objects so that when one object (the subject) changes state, all its dependents (observers) are notified and updated automatically.

### Key Characteristics

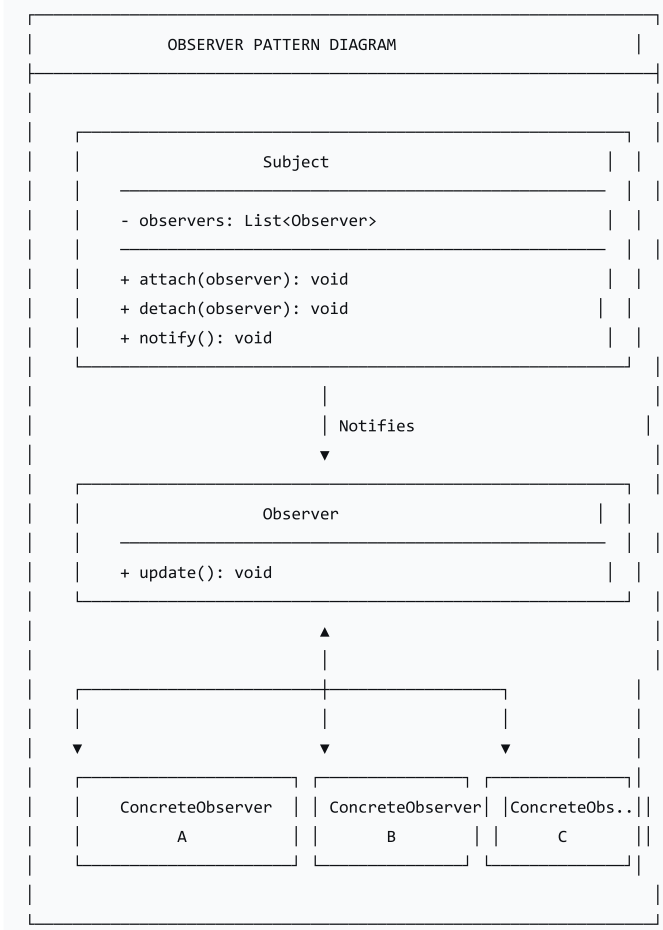
| Characteristic | Description |
|---|---|
| **One-to-Many** | One subject can have many observers |
| **Automatic Notification** | Observers are notified of state changes |
| **Loose Coupling** | Subject and observers are loosely coupled |
| **Push/Pull** | Can push data to observers or allow them to pull |

## When to Use Observer

| Scenario | Why Observer? |
|---|---|
| **GUI Event Handling** | Button clicks notify listeners |
| **News Subscription** | Subscribers get notified of new content |
| **Stock Market Updates** | Traders get notified of price changes |
| **Weather Station** | Weather data updates multiple displays |

## Observer Pattern Diagram

text




## Conceptual Implementation

python

```python
# Python
class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer)

    def detach(self, observer):
        self._observers.remove(observer)

    def notify(self, message):
        for observer in self._observers:
            observer.update(message)

class Observer:
    def update(self, message):
        pass

class EmailObserver(Observer):
    def update(self, message):
        print(f"Email notification: {message}")

class SMSObserver(Observer):
    def update(self, message):
        print(f"SMS notification: {message}")

class PushObserver(Observer):
    def update(self, message):
        print(f"Push notification: {message}")

# Usage
subject = Subject()
email = EmailObserver()
sms = SMSObserver()
push = PushObserver()

subject.attach(email)
subject.attach(sms)
subject.attach(push)

subject.notify("New message received!")
```

java

```java
// Java
import java.util.ArrayList;
import java.util.List;

interface Observer {
    void update(String message);
}

class Subject {
    private List<Observer> observers = new ArrayList<>();

    public void attach(Observer observer) {
        observers.add(observer);
    }

    public void detach(Observer observer) {
        observers.remove(observer);
    }
```

```
    public void notify(String message) {
        for (Observer observer : observers) {
            observer.update(message);
        }
    }
}

class EmailObserver implements Observer {
    public void update(String message) {
        System.out.println("Email notification: " + message);
    }
}

class SMSObserver implements Observer {
    public void update(String message) {
        System.out.println("SMS notification: " + message);
    }
}

// Usage
Subject subject = new Subject();
subject.attach(new EmailObserver());
subject.attach(new SMSObserver());
subject.notify("New message!");
```

---

## 6. Design Patterns Summary

### Comparison Table

| Pattern | Type | Purpose | Key Feature |
|---|---|---|---|
| **Singleton** | Creational | Ensure a single instance | Private constructor, getInstance() |
| **Factory** | Creational | Create objects without specifying class | Interface/abstract class, concrete creators |
| **Observer** | Behavioural | Notify dependents of state changes | Subject, observers, update method |

### Pattern Use Cases at a Glance

| Pattern | When to Use | Example |
|---|---|---|
| **Singleton** | Need one instance globally | Configuration, logging, database connections |
| **Factory** | Need flexible object creation | Payment systems, UI components |
| **Observer** | Need to notify multiple objects | Event handling, news feeds, stock updates |

---

## 7. Worksheet Activities

### Activity 1: Identifying Design Patterns

**Scenario 1:** A system that needs to ensure only one instance of a configuration manager exists.

**Answer:** Singleton Pattern

**Scenario 2:** A system that needs to create different types of documents (PDF, Word, HTML) without tightly coupling the client.

**Answer:** Factory Pattern

**Scenario 3:** A stock market application where investors need to be notified when stock prices change.

**Answer:** Observer Pattern

---

## 8. Lesson Sequence

### Lesson Plan (60 minutes)

| Phase | Time | Activity |
|---|---|---|
| **Introduction** | 5 min | Hook: Recurring problems; architectural analogy |
| **Design Patterns** | 10 min | Definition, benefits |
| **Singleton** | 15 min | Explanation, use cases, conceptual implementation |
| **Factory** | 15 min | Explanation, use cases, conceptual implementation |
| **Observer** | 10 min | Explanation, use cases, conceptual implementation |
| **Wrap-up** | 5 min | Review; worksheet introduction |

---

## 9. Assessment Activities

| Assessment Type | Description |
|---|---|
| **Class Participation** | Observe engagement and contributions |
| **Worksheet** | Evaluate understanding of design patterns |
| **Coding Worksheet** | Assess ability to apply design patterns in code |

---

## 10. Differentiation

| Level | Strategy |
|---|---|
| **Support** | Use analogies; break down code examples |
| **Challenge** | Explore additional patterns (Strategy, Decorator) |

---

## 11. Resources

- **Presentation** (Slide Deck)
- **eBook** (B3.2.5 with examples)
- **Worksheets:** Design pattern identification and coding
- **Worksheet Answers**

---

## 12. Summary: Key Takeaways

| Pattern | Purpose | Key Idea |
|---|---|---|
| **Singleton** | Single instance | One instance, global access |
| **Factory** | Object creation | Decouple client from concrete classes |
| **Observer** | Notify dependents | One-to-many notification |

```text
┌──────────────────────────────────────────────────────────┐
│                   KEY REMINDERS                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│    ✅ Design patterns = reusable solutions               │
│    ✅ Singleton = one instance only                      │
│    ✅ Factory = flexible object creation                 │
│    ✅ Observer = notify many dependents                  │
│    ✅ Patterns improve reusability and maintainability   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

> *"Design patterns are proven solutions to common problems—they help developers write more maintainable, reusable, and understandable code."*